

Trading Engine – Developer Specification (Live, Config-Driven)

It is a **generic options engine** – all strategy logic, thresholds, and conditions will come from configuration files, not from hard-coded rules.

1. High-Level Objective

Build a modular trading engine that can:

1. Connect to our broker/platform (e.g. IBKR) for **live** data:
 - Index/underlying price data.
 - Options data, **including** IV and delta.
2. Evaluate **technical conditions** on the underlying:
 - Opening range breakout.
 - Moving average / trend filters.
3. **Fetch volatility data**:
 - IV of strikes(ATM & relevant strikes) and IV percentile (IVP).
4. When conditions are met, build **multi-leg option structures** such as:
 - Vertical spreads (credit/debit).
 - Iron condors.
 - Iron flies.
 - Straddles/strangles.
 - Ratio spreads.
5. Manage open positions:
 - Monitor spread value every N minutes.
 - Close on profit target, stop loss, or a cut-off time.

6. Be **fully controlled by CSV configuration**:

- All thresholds and toggles are in CSV.
 - All strategy templates (which legs, which deltas, which expiries, etc.) defined in CSV.
 - No strategy-specific constants baked into the code.
-

2. Modes & Architecture

2.1 Two modes

The engine must work in following mode:

1. **Live mode**

- Data comes from:
 - Broker (e.g. IBKR) real-time and historical API.
- Run strategy live and log trades.

2.2 Modules

Please structure the engine logically into:

1. **Data layer**

- Underlying (index) data fetch.
- Options data fetch (chains, quotes, IV, delta).

2. **Condition/indicator layer**

- Computes price-based signals (breakouts, moving averages, etc.).
- Read volatility info (IV, IVP).
- Outputs simple flags/labels like `direction_bias`, `vol_regime`, `trend_regime`, `signal_ok`.

3. **Strategy/structure layer**

- Chooses a **strategy template** based on labels (direction, vol regime, etc.) and config mapping.
- Builds a specific multi-leg structure using **delta-based leg definitions** from config.

4. Risk & monitoring layer

- Calculates spread entry value.
- Computes target & stop based on config factors.
- Monitors position every N minutes and triggers exits.

5. Config & logging layer

- Loads CSV configs into structured objects.
- Writes trade logs and optional debug/signal logs.

3. Data Requirements (Broker Integration)

3.1 Underlying (Index) Data

We need functions that, given a **date** (in backtest) or in **real-time** (live):

- Fetch intraday bars for the underlying index:
 - Time, open, high, low, close.
 - Configurable bar size (e.g. 1-minute, 5-minute, 10-minute).

Configuration examples:

- `underlying_symbol` (e.g. SPXW, CBOE, ES, CME)
- `bar_interval_minutes` (e.g. 1, 5, 10).
- `session_start_time` (e.g. 09:30).
- `session_end_time` (e.g. 16:00).

3.2 Options Data

We need functions that can, for a given **underlying**,:

- Discover option contracts by:
 - Expiry (e.g. nearest expiry, same-day).

- Strike (closest to spot or by rule).
 - Type: call or put.
- Fetch fields for a specific option contract:
 - Price (we'll use a configurable field: last/mid/bid/ask).
 - **Implied volatility**.
 - **Delta** (and other greeks if needed).
 - Expiry, strike, type (call/put).

Both IV and delta should be **directly fetched from the broker data**.

Configuration examples:

- `option_entry_price_field` – "last", "mid", etc.
- `use_broker_iv` – YES/NO.
- `use_broker_delta` – YES/NO.
- `default_expiry_rule` – e.g. "same_day" or "nearest_weekly".

4. Technical Logic (Price-Based Conditions)

We want the engine to support a few basic technical modules. The exact thresholds and whether they're used will come from config.

4.1 Opening Range (ORB)

Concept:

- **Opening range**: first N minutes after session start.
- Compute:
 - `OR_low` = minimum low in that range.
 - `OR_high` = maximum high in that range.

After some start time (e.g. 10:45), for each subsequent bar:

Position Building Logic based on Bar Close:

Condition	Position to be Built
Bar close is above the Opening Range High (OR_high)	Long
Bar close is below the Opening Range Low (OR_low)	Short
Bar stays within the Opening Range until a timeout	Neutral

CSV parameters:

- `use_orb` – YES/NO.
- `orb_window_minutes` – length of opening range (e.g. 30 or 60 minutes, multiple of 5).
- `orb_start_time` – e.g. 09:30.
- `orb_breakout_start_time` – e.g. 10:30.
- `orb_breakout_end_time` (can be used as a timeout to play neutral view– e.g. 12:00).

Outputs (internal labels):

- `direction_bias` $\in$ {BULLISH, BEARISH, NEUTRAL}.
- `orb_breach_time` and `orb_breach_price` for logging.

4.2 Trend Filters

We also want generic support for trend filters such as:

- Short moving average vs long moving average (for underlying close):
 - e.g. 50-period vs 100-period.
- Price vs moving average:
 - e.g. latest close > MA_short, etc.

CSV parameters:

- `use_trend_filter` – YES/NO.
- `ma_short_length` – e.g. 50.
- `ma_long_length` – e.g. 100.

- `trend_mode` – e.g.:
 - `confirm_with_trend` (only trade if bias agrees with trend),
 - `ignore_trend`, etc.

Outputs:

- `trend_regime` $\in$ {`UPTREND`, `DOWNTREND`, `SIDEWAYS`}.
- `trend_ok` – boolean decide if trading is allowed.

4.3 Combined Direction Logic

Final direction can be a function of ORB and trend (config-driven), e.g.:

- `direction_source` =
 - `"orb_only"`,
 - `"trend_only"`,
 - `"orb_and_trend_agree"` (ex: trade only if both agree).
 - `" if within range (within OR)"`

This is not hard-coded logic; the developer just needs to support these selection modes via config.

5. Volatility Logic (IV, IVP, VRP, VRT)

We will use the broker for IV and IVP.

5.1 IV & Baseline Time

We want to evaluate volatility at one or more **reference times** during the session (for example, around 10:00–10:15).

At each reference time:

- Get the underlying spot.
- Define an “ATM” strike (e.g. nearest strike).
- Get the IV for ATM call/put (or directly from ATM straddle).

CSV parameters:

- `iv_reference_times` – e.g. `10:05`, `10:10` (we can support multiple).
- `atm_strike_rounding_step` – e.g. `5`.
- `iv_smoothing_method` – e.g. average call/put or single side.

Store these IVs.

5.2 IV Percentile (IVP)

We want a flexible way to assign a **vol regime** using IVP:

- Maintain the IVP fetched from the broker.

CSV parameters:

- `use_ivp` – YES/NO.
- `ivp_window_days` – history length.
- `ivp_1_threshold` – (e.g. `40`).
- `ivp_2_threshold` – (e.g. `50`).

Outputs:

- `vol_regime` $\in \{\text{vol}_1, \text{vol}_2, \text{vol}_3, \}$

6. Strategy Selection (Generic Mapping, Hidden Logic)

we want:

1. Trader will use one or more of the below condition to generate triggers:
 - `direction_bias` (BULLISH/BEARISH/NEUTRAL).
 - `vol_regime` (LOW/MID/HIGH).
 - `trend_regime` (UP/DOWN/SIDEWAYS).
 - plus some generic flags like `custom_flag_1`, `custom_flag_2` (we will use internally).
 - Basis direction bias and/or vol and trend regime, trader to select which strategy or multi-leg structure to be deployed.

2. A **CSV-driven mapping** that says:
 - For each combination of these labels, which **strategy template ID** is allowed, or highest-priority.

Example (conceptual):

direction	vol_regime	trend_regime	strategy_id
BULLISH	vol_1	UPTREND	STRAT_CREDIT_PUT
BEARISH	vol_1	DOWNTREND	STRAT_CREDIT_CALL
ANY	vol_2	ANY	STRAT_IRON_CONDOR

7. Strategy Templates & Delta-Based Legs

Each **strategy template** describes a multi-leg structure. Examples:

- Credit vertical spread.
- Debit vertical spread.
- Iron condor.
- Iron fly.
- Straddle/strangle.
- Ratio spread.

7.1 Leg Specifications (Key Part)

Each leg must be defined in a template by:

- **leg_index** – 1, 2, 3, ...
- **option_type** – "C" (call) or "P" (put).
- **side** – "BUY" or "SELL".
- **target_delta_abs** – target absolute delta (e.g. 0.25, 0.10, 0.50).
 - The engine finds the actual strike whose **delta is closest** to this value.
- **delta_side** (optional) – to express call/put sign if needed.
- **expiry_rule** – e.g. same-day, nearest, +1 day, etc. (initially can just be "nearest expiry").
- **quantity_factor** – 1, 2, etc. to allow ratio spreads.

The **strike selection is always delta-based**:

- Engine pulls the option chain, with IV and delta for each strike.
- Filters by expiry rule.
- For each leg, finds the option whose $|\text{delta}|$ is closest to `target_delta_abs`.

CSV example columns for strategy template:

- `strategy_id`
- `leg_index`
- `option_type`
- `side`
- `target_delta_abs`
- `expiry_rule`
- `quantity_factor`

7.2 Strike Search Parameters

Engine needs some default search settings:

- `strike_step` – minimum strike gap.
- `max_strike_range_points` – how far away from spot we can search.
- `strike_search_expand_step_points` – step for widening search if target delta cannot be found close to spot.

These **must be configurable**.

8. Trade Entry, Risk, Monitoring & Exit

8.1 Entry Time & Price

Entry time:

- Configurable approach; a typical pattern is:

- Once conditions are met (e.g. ORB + filters), **entry time = first bar close that triggers** plus a configurable offset (few minutes).
- Config parameters:
 - `entry_time_mode` – e.g. “on_signal_close_plus_offset”.
 - `entry_offset_minutes` – offset in minutes.

Entry prices:

- For each leg, we fetch market prices from broker.
- Use a **configurable price field** and slippage, e.g.:
 - Entry price = spread_price (We will place orders via midprice of spreads):

CSV parameters:

- `entry_price_field` – “last”, “mid”, etc.
- `slippage_per_leg` – fixed amount per option.

8.2 Spread Value & Target/Stop

Spread value:

- Define a convention:
 - $\text{spread_value} = \Sigma(\text{buy_leg_prices}) - \Sigma(\text{sell_leg_prices})$

Credit structures:

- `spread_value` is negative initially (receive premium).

Debit structures:

- `spread_value` is positive initially (pay premium).

We define **target and stop** as **configurable functions** of entry spread:

Config example:

- For credit spreads:
 - `credit_target_factor` (e.g. 0.5) – target is some fraction towards zero (spread shrink).

- `credit_stop_factor` (e.g. 1.5) – adverse move factor.
- For debit spreads:
 - `debit_target_factor` (e.g. >1) – target is multiple of entry.
 - `debit_stop_factor` (e.g. <1) – cut at fraction.

8.3 Monitoring

Engine must monitor open positions every N minutes:

1. At each checkpoint:
 - Fetch latest prices for all legs.
 - Compute current `spread_value` (without slippage).
 - Optionally, cap it within theoretical min/max bounds (based on strikes).
2. Compare with `target` and `stop`:
 - If target hit → close position, exit reason `TARGET`.
 - If stop hit → close position, exit reason `STOP`.
3. If neither hit by a certain time (cutoff), close by **time exit**:
 - Use final prices with slippage.
 - Exit reason `TIME`.

Config parameters:

- `monitor_interval_minutes`.
- `cutoff_time` (e.g. 15:30).
- `exit_on_target`, `exit_on_stop`, `exit_on_time` – toggles.

8.4 Position Sizing

We need flexible sizing:

Config parameters:

- `max_cap_to_be_used`
- `contracts_count` ≤ int(max_cap_to_be_used/margin_required for 1 lot).
- `contract_size` – e.g. 100.

Note: Margin can be fetched directly from brokers platform.

9. Configuration CSV – Must Control Everything

We want **no hidden constants** in code.

At minimum, configuration must cover:

1. Symbol

- Underlying symbol.

2. Times & ORB

- Session times.
- ORB start/end.
- Breakout window.
- Entry offset.
- Monitor interval.
- Cutoff time.

3. Technical filters

- Use ORB YES/NO.
- MA lengths.
- Trend filter toggles.
- Mode for combining ORB & trend.

4. Volatility filters

- Use IVP YES/NO.
- IVP thresholds.

5. Strategy mapping

- Rows describing (`direction`, `vol_regime`, `trend_regime`, `custom_flags`) → `strategy_id`

6. Strategy templates

- For each `strategy_id`:
 - Leg definitions with delta targets, side, expiry rule, quantity, etc.

7. Risk & sizing

- Target/stop factors (global and per-strategy).
- Position size parameters.

8. Logging

- Log paths.
- Verbosity flags.

- Log signals as well as trades.
-

10. Logging & Outputs

The engine should always produce:

1. Trade log CSV

- Strategy ID.
- Date/time.
- Direction & regimes (labels).
- Leg details (type, strike, side, delta target vs actual).
- Entry/exit spread.
- Target/stop values.
- Exit reason.
- PnL per contract & total.

2. Optional signal / diagnostics CSV

- For debugging and tuning, e.g.:
 - Underlying price, ORB, MA, IV, IVP..
 - Which conditions passed or failed.
 - Which strategy (if any) was selected.
-